

INFO 579 Week 7 Final Project Report

Course: INFO 579: SQL/NoSQL Databases for Data and Information Sciences

Module/Week: 7 - Week of August 18, 2025

Topic: Week 7 Final Project Report

NOTE: The assignment document must provide the following information. Up to 5 points may be deducted due to the lack of the below information.

Student's Full Name: Ripa Shah

Course Title: INFO 579: SQL/NOSQL Databases for Data and Information Sciences

Term name and year: Example, Summer 2025

Submission Week: [example, Week 7 Final Project Report]

Instructor's Name: N. Rahman

Date of Submission: 8/23/2025

The above information must be provided at the upper left corner of the first page of the document.

Each answer(s) must be preceded by the question/ title of the topic/article of the assignment.

Acceptable File: PDF.

File Name Format: Name your file according to this convention: INFO579_Final Project_Report_Lastname.pdf. **Submission must be made in a single document.**

NOTE: A 20% deduction of points will be applied for any late submission.

Week 7 Final Project Report: Total Points – 100

Goodwill Store Database

Abstract:

The Goodwill store doesn't have a database system to manage the inventory, transaction and store information so I've gathered Goodwill store information by going into store physically. Also, I've gathered a little information from the online website. After that, I'm able to create a normalized data for Goodwill store. As a result, I'm able to store the information into Goodwill store database related to all day to day transactions, inventory, sales and sponsors etc.

Introduction:

The Goodwill store is near my home, so I visit it often. I also interact with them. After interacting with them, I figured it out that Goodwill store would like to organize its data by using a relational database. By this way, they can manage goods branch wise. As well as they can see how many items are sold from the store. Along with that, they can keep track of their partners and program services. In this report, I've gathered Goodwill store information and create a database and tables.

It is crucial to understand foundation before we implement the Goodwill Store project so let me walk you through some basic concepts which will be beneficial to read entire project report.

Relational Data model:

Relational models are a graphical representation of relational database where it depicts how it will be managed and stored into the relational database. There are three types of relational data models:

- (1) **Conceptual data model:** It is a high level of business model where data are not normalized yet. It consists of entities and attributes. Entities are something about which we store the data. And attributes are characteristics of entities.
- (2) **Logical data model:** A logical model provides detailed description of entities, attributes and constraints including foreign keys and primary keys. In this model, it demonstrates the graph in a normalization form.
- (3) **Physical data model:** A Physical data model is similar to logical data model. In addition to logical model, it provides attributes data type, which is helpful while building a table in any relational database language including SQL, MySQL, PostgreSQL, and Oracle. It will convert entities into tables. Attributes into columns along with data types, and relationships into foreign keys/ constraints.

To follow physical data models, we must have a basic idea about the normalization concepts:

Normalization: Normalization is a design technique which eliminates data redundancy. It divides the larger tables into smaller tables and links them with relationships. The purpose of normalization is to eliminate redundancy from the table and maintain atomicity. I'm using following example to explain normalization concept. It's a video library that maintains data without any normalization which movies have rented out.

FULL NAMES	PHYSICAL ADDRESS	MOVIES RENTED	SALUTATION
Janet Jones	First Street Plot No 4	Pirates of the Caribbean, Clash of the Titans	Ms.
Robert Phil	3 rd Street 34	Forgetting Sarah Marshal, Daddy's Little Girls	Mr.
Robert Phil	5 th Avenue	Clash of the Titans	Mr.

1NF: It ensures that database tables contain an atomic values and each record is unique.

Example:

1NF Example

FULL NAMES	PHYSICAL ADDRESS	MOVIES RENTED	SALUTATION
Janet Jones	First Street Plot No 4	Pirates of the Caribbean	Ms.
Janet Jones	First Street Plot No 4	Clash of the Titans	Ms.
Robert Phil	3 rd Street 34	Forgetting Sarah Marshal	Mr.
Robert Phil	3 rd Street 34	Daddy's Little Girls	Mr.
Robert Phil	5 th Avenue	Clash of the Titans	Mr.

2NF: It must be in 1NF. There is no partial dependency.

Partial dependency means a single Column Primary Key that is not functionally dependent on any subset of candidate key relation.

Example:

MEMBERSHIP ID	FULL NAMES	PHYSICAL ADDRESS	SALUTATION
1	Janet Jones	First Street Plot No 4	Ms.
2	Robert Phil	3 rd Street 34	Mr.
3	Robert Phil	5 th Avenue	Mr.

MEMBERSHIP ID	MOVIES RENTED
1	Pirates of the Caribbean
1	Clash of the Titans
2	Forgetting Sarah Marshal
2	Daddy's Little Girls
3	Clash of the Titans

3NF: It must be in 2NF. There is no transitive dependency.

MEMBERSHIP ID	FULL NAMES	PHYSICAL ADDRESS	SALUTATION ID
1	Janet Jones	First Street Plot No 4	2
2	Robert Phil	3 rd Street 34	1
3	Robert Phil	5 th Avenue	1

MEMBERSHIP ID	MOVIES RENTED
1	Pirates of the Caribbean
1	Clash of the Titans
2	Forgetting Sarah Marshal
2	Daddy's Little Girls
3	Clash of the Titans

SALUTATION ID	SALUTATION
1	Mr.
2	Ms.
3	Mrs.
4	Dr.

A transitive functional dependency means changing a one non key column may cause changing in another non-key column. Thus, we've created another salutation table to remove transitive dependency in above example.

Primary Key: A single column value used to identify a database record uniquely. Primary key cannot be null. It must be unique.

Foreign Key: It ensures that rows in one table have corresponding rows in another.

Composite Key: A primary key composed of multiple columns used to identify a record uniquely.

I've gathered Goodwill store data. By collecting that information, I've designed conceptual, logical and physical models. To implement this project, I've utilized Azure PostgreSQL Server functionality. In this report, I've implemented following activities for Goodwill store.

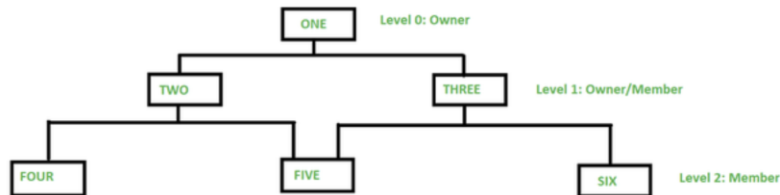
- (1) Data model – (Physical, Logical and conceptual model)
- (2) Implemented normalization concepts
- (3) Applied constraints to Goodwill tables

- (4) Insert the data into Goodwill tables.
- (5) Create a variety of SQL queries to retrieve data from one or many tables

Related Work:

Hierarchical model: The hierarchical model stores data in a tree-like structure form. Windows registry is one of the best examples of hierarchical models. The hierarchical structure was developed by IBM in the 1960s and used in early mainframe DBMS. Records' relationships form a tree-like model. It has a first node called a root node. It is a kind of parent-child relationship. The root node at the peak depicts the beginning, and branches facing downwards to child nodes, exemplifying the relationships and route of data access. Active directory, File system and geographic information system is an example of hierarchical model. Hierarchical models have a few limitations including data rigidity, data redundancy, complex implementation which will be cured by relational database management system due to its normalized structure.

Network Database Model:



A Network data model

In the 1960s, the Database Task group established this paradigm. The hierarchical model is generalized in this model. There is a logical connection between the segments that belong to each level, even if this model may have several parent segments that are grouped as levels. Most of the time, any two segments have a reasonable many-to-many relationship. The logical relationships between the segments are referred to as graphs. Consequently, this model substitutes a graph-like structure for the hierarchical tree, allowing for more general linkages between various nodes. A record can have more than one parent section thanks to M: N relations, or many-to-many.

Week 2 Final Project Update 1: Total Points - 25

Week 2 Final Project Update 1:

1. Come up with a business plan for which you need to collect and store data
2. Come up with the data about your business. Example: like the one I gave you for the assignment.

Requirements: As part of the **Final Project Update 1**, submit a single-spaced one-page report:

1. A brief description of your project

A goodwill store would like to organize its data by using a relational database. By this way, they can manage goods branchwise. As well as, they can see how many items are sold from the store. Along with that, they can keep a track of their partners and program services. Here I'm providing data to implement a detailed relational management system.

2. Data you will use and source of that data:

Source: <https://www.goodwill.org/>

(1) Goodwill branch / store location

Goodwill is a franchise. It has a branches all over to the USA.

GoodWill BranchId	Location	City	Zipcode	State
GW1	5526 Renner Rd	Columbus	43228	OH
GW2	2550 North High St	Columbus	43202	OH
GW3	608 Main St,	Hilliard	43026	OH

(2) Goodwill Employee Information:

Employee information who works for Goodwill

EmployeeId	FirstName	LastName	Position	BranchId	HireDate	Phone
Emp1	John	Johnson	Manager	GW2	03/02/2021 456-456-6	788

Emp2	Joshua	Soboti	Cashier	GW1	04/02/2024 345-345-6	789
Emp3	Ashlee	Nimmo	Cashier	GW3	05/06/2025 908-890-7	890

(3) Goodwill Donor information:

Individuals or organizations providing goods to Goodwill

DonorId	Donar Name	Item Received	Date
Donor01	John	Clothes	03/02/2023
Donor02	Kelly McClure	furniture	04/05/2021
Donor03	Jessica Mendez	miscellaneous	05/03/2020
Donor04	Amber Hammer	clothes	06/02/2024

(4) Warehouse:

Where donations are sorted, cleaned, repaired and priced.

DatawarehouseID	Location	Capacity	ProcessingStaffcount
DW01	Columbus	1000	40
DW02	Michigan	1000	50

(4) Goodwill Donation information

The goods contributed by donor.

DonationId	Item Received	Date Received	Quantity	Condition	BranchId
Donation01	Clothes	03/04/2023	10	Good	GD01
Donation02	Furniture	02/03/2023	5	Fair	GD01

(6) Item:

Individual salable goods after processing.

ItemId	Category	Subcatego	Price	Status	DatePriced	BranchId
--------	----------	-----------	-------	--------	------------	----------

		ry				
It01	Clothing	Men's apparel	10	In Stock, Sold, Recycled	01/03/2025	GD01

(6) Customer:

Individual who purchases items from goodwill store.

CustomerId	PurchaseHistory	Item purchased	Transaction details
Cust01	01/03/2024	it01	30
Cust02	02/04/2025	it02	40

(7) Transaction:

Records of sales made at the stores

TrasactionId	Date Of Sale	Total Amount	Payment Method	CustomerId	BranchId
Tr01	04/03/2025	400	Visa Credit	Cust01	GW1
Tr02	05/02/2024	450	Master Credit	Cust02	GW2

(8) Program / Service (Community Impact):

The social programs and services funded by Goodwill's retail operations (e.g., job training, career services).

ProgramId	Program Name	Description	Target Beneficiaries	Funding Source
Pr01	Job training		Job seekers	Sp01
Pr02	Career Services		Job seekers	SP02

(9) Sponsor:

provide financial support. Donate goods or services. Raise awareness. Collaborate on training programs. Support community events.

SponsorId	Sponsor Name	Sponsor Type
Sp01	amazon	Targeted Sponsors
Sp02	comcast	Enterprise Solution Partners

(11) Item subCategory

Subcategory	Category	Item details
sub01	Men's apparel	Clothes, jackets, pants
sub02	Women's apparel	Clothes, blouses, jackets, pants
sub03	Furniture	Couches, chairs, dining tables, tables

3. Upload the data file (like the one I gave you to do the assignments).

I have uploaded an excel document.

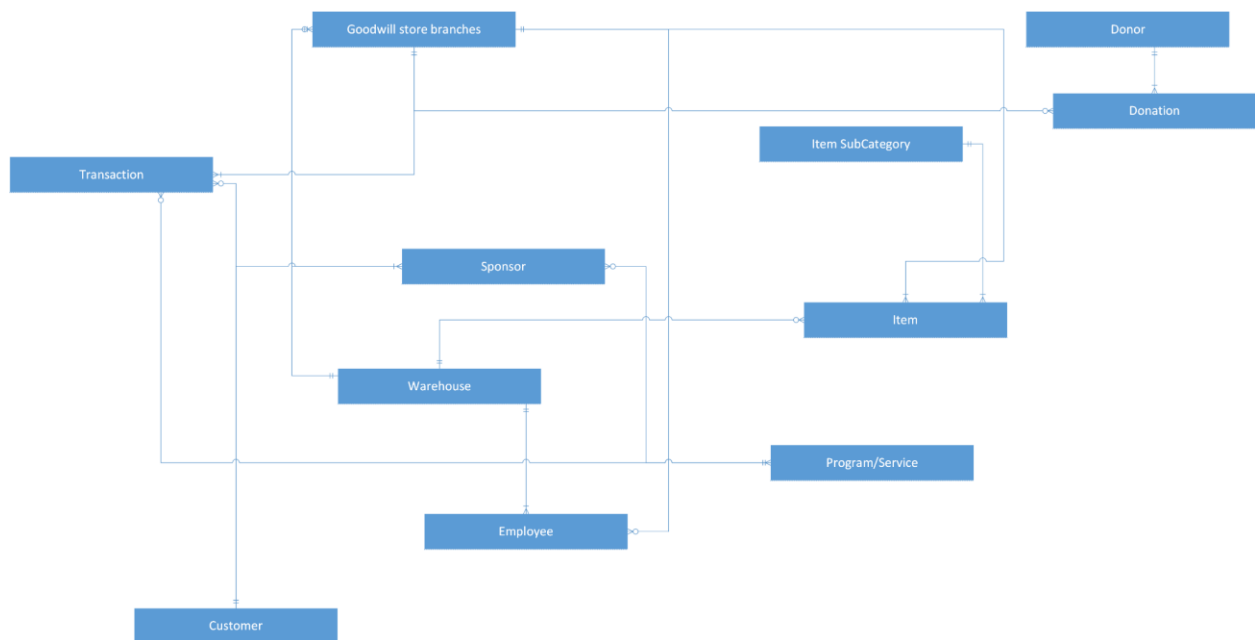
Week 4 Final Project Update 2: Total Points - 25

3. Develop a Conceptual Model. Consider 4 or 5 entities. Make sure you have at least one many-to-many relationship. Explain with data why it's a many-to-many relationship.

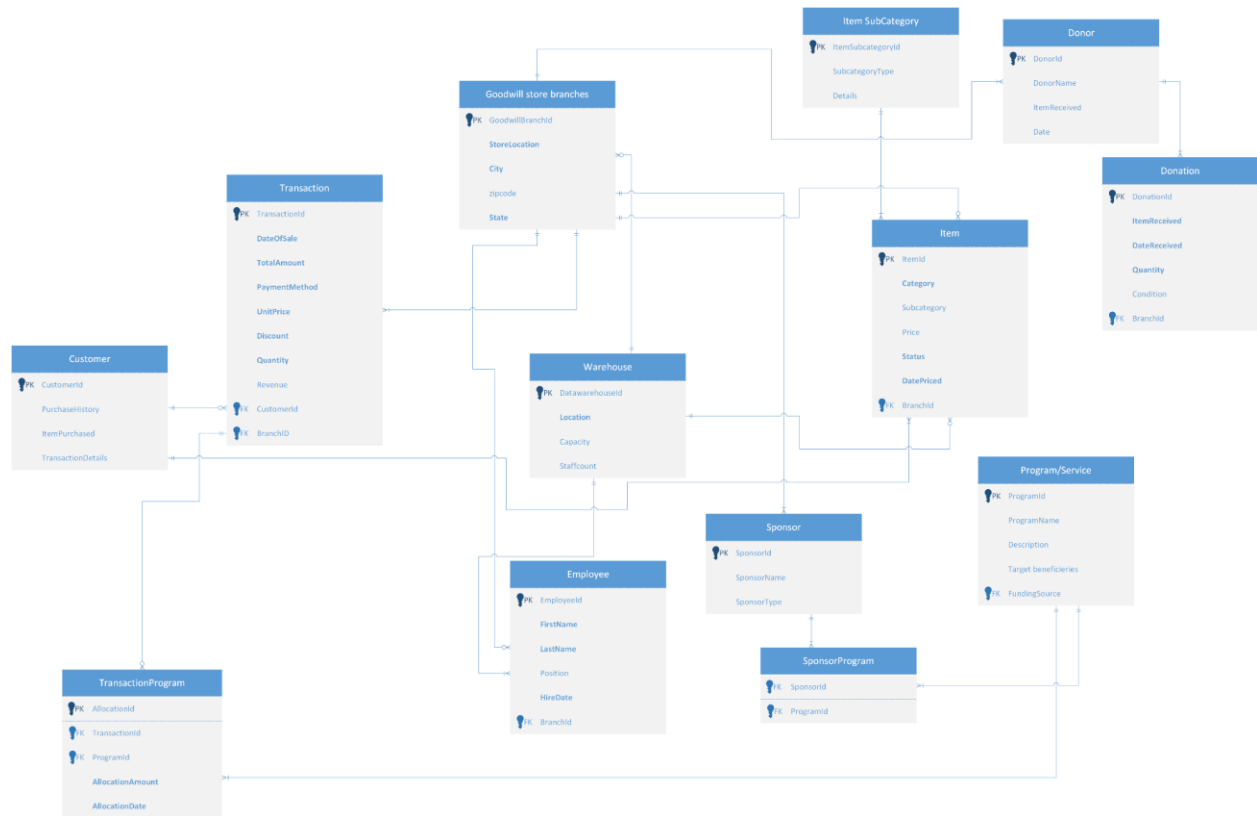
Start by capturing the main objects of interest:

- **Donor** – Individuals donating items

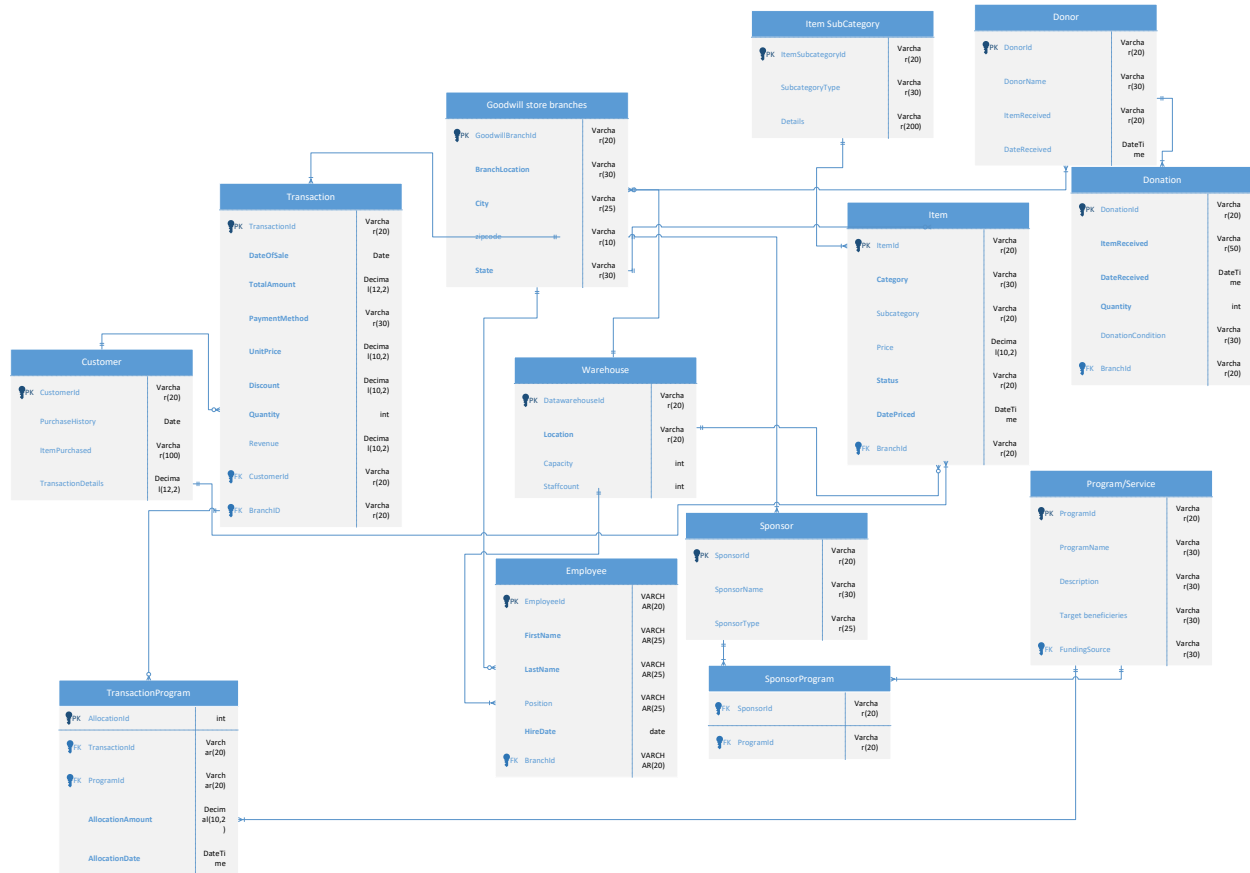
- **Donation** – Details about each donation
- **Item** – Individual items donated
- **Store Location** – Where items are sold
- **Warehouse** - where donation are sorted, cleaned, priced and repaired.
- **Transaction** - Records of sales made at the store.
- **Customer** – Buyers of items
- **Employee** – Staff who manage operations
- **Category** – Item classification (e.g. clothing, furniture)
- **Program/ Service** - The social programs and services funded by goodwill's retail operations.
- **Sponsor**- sponsors who donates in support of running programs and services.



4. Develop a Logical Model using the Conceptual Model. Make sure you come up with a junction entity to resolve the many-to-many relationship.



5. Develop the physical model based on the Logical Model



Once I'm able to create PostgreSQL in azure. I'll install editor to work with PostgreSQL using [PostgreSQL: Downloads](#)

Now I've install PostgreSQL-PGAdmin4 on my machine. Also, I'm able to connect Azure Postgres with PGAdmin4.

St

goodwillstore.postgres.database.azure.com

×

General

Connection

Parameters

SSH Tunnel

Advanced

Post Connection SQL

Tags

Host name/address

goodwillstore.postgres.database.azure.com

Port

5432

Maintenance database

postgres

Username

postgreuser

Kerberos authentication?

☐

Password

In edit mode the password field is enabled only if Save Password is set to true.

Save password?

☐

Role

Service

i

?

×

Close

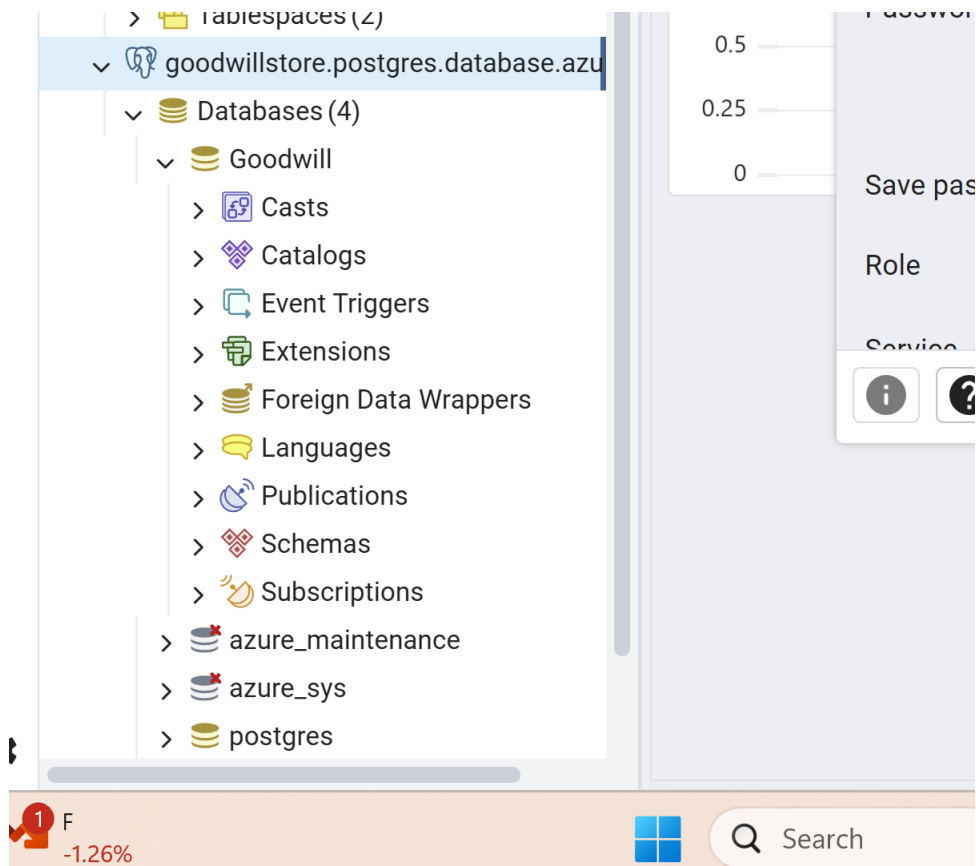
↺

Reset

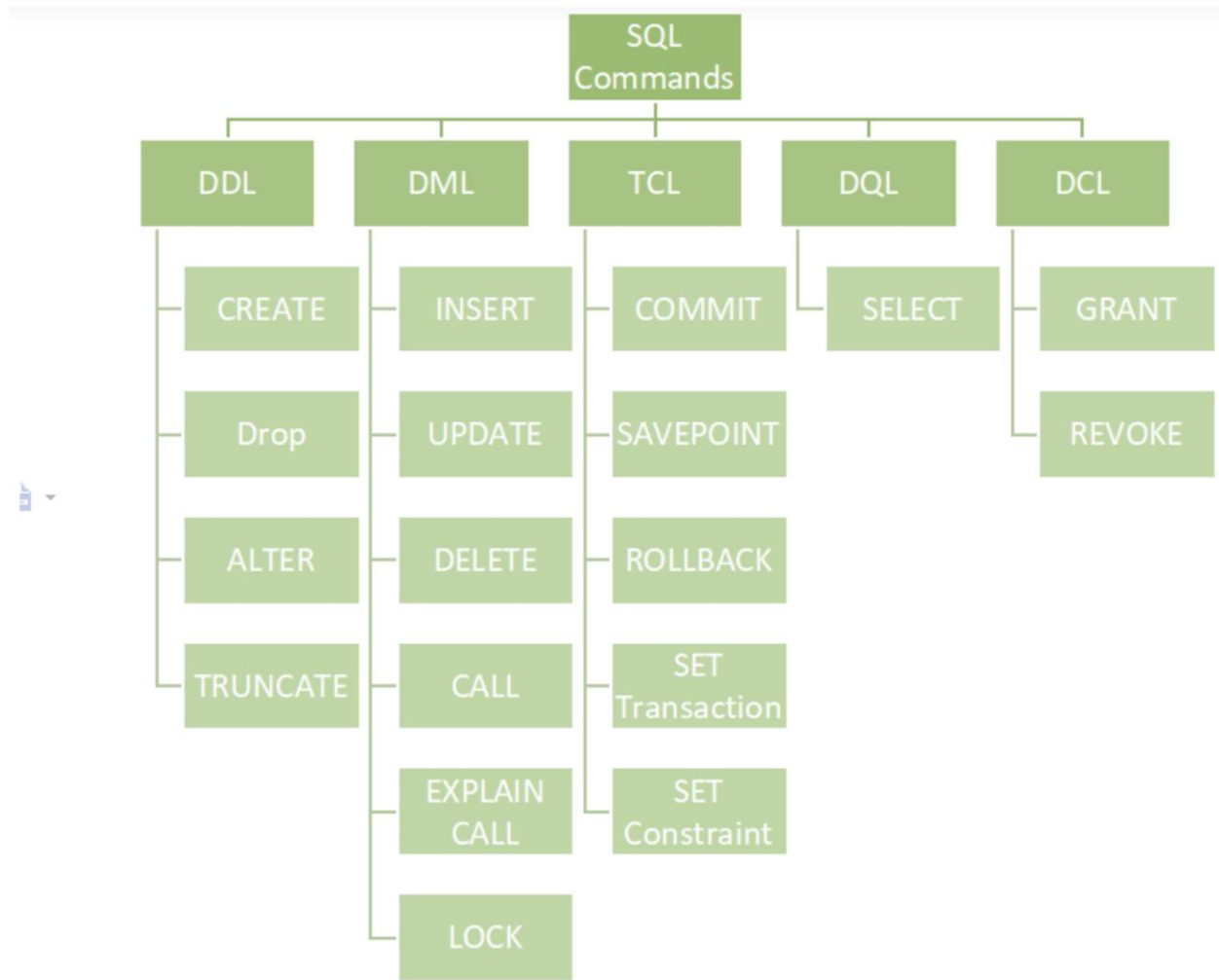
💾

Save

I can also populate the databases which I've created in azure.

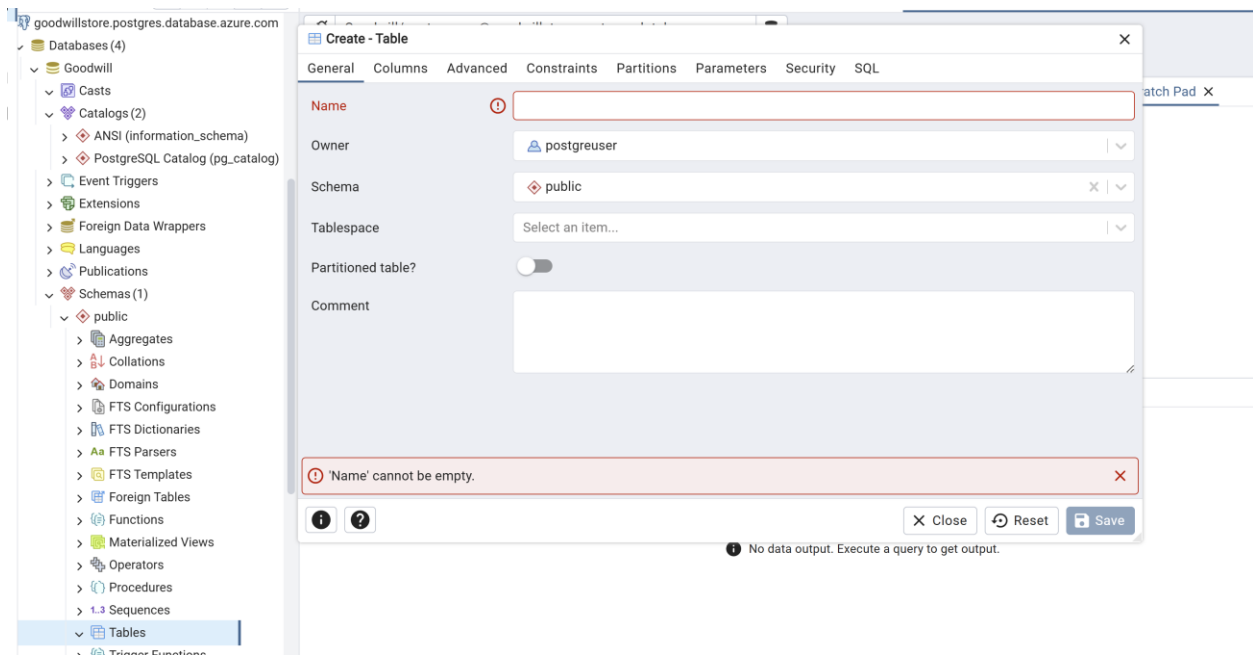


Now, let me focus on creating tables that I've utilized in my physical data model. By this way, I'm able to implement database normalization concepts. Along with that, I'm able to perform DML (Data manipulation Language), DQL (Data Query Language) and DDL (Data definition Language).



So let's start with a DDL. There are two options we can create tables. (1) GUI (2) Script

Here, I've taken a screenshot of GUI. I believe it's more cumbersome than script option so let me choose script instead of choosing GUI since you just need to go step by step and it's more time consuming than script.



Here is my create table option. We'll utilize this option throughout creating a table from physical data model.

The screenshot shows a web-based PostgreSQL client interface. At the top, there are tabs for 'Dependencies', 'Dependents', 'Processes', and several database connections including 'Goodwill/postgreu...', 'postgres/postgreu...', and 'Goodwill/postgreuser'. Below the tabs is a toolbar with icons for file operations, filters, and execution. The main area is divided into 'Query' and 'Query History' tabs. The 'Query' tab contains a SQL statement to create a table named 'Goodwill_Branch' with columns: 'Branchid' (int, primary key), 'BranchLocation' (varchar(30)), 'City' (varchar(25)), 'Zipcode' (varchar(10)), and 'state' (varchar(20)). Below the query editor, there are tabs for 'Data Output', 'Messages', and 'Notifications'. The 'Messages' tab is active and displays the text 'CREATE TABLE' and 'Query returned successfully in 248 msec.'

```
1 create table Goodwill_Branch
2 (Branchid int primary key,
3 BranchLocation varchar(30),
4 City varchar(25),
5 Zipcode varchar(10),
6 state varchar(20));
```

CREATE TABLE

Query returned successfully in 248 msec.

6. Create tables using a database system. Insert data into the database tables. You must provide the DDL (CREATE TABLE statements), INSERT statements, and SELECT statements.

Details: Create the tables that you have come up with (the table must be based on the Physical Model).

(a) Columns, Primary Key (PK), Data Type, and length, and NULL/NOT NULL need to be implemented, per the Physical Model.

(b) Show the table definition (DDL) that you implemented (not in a graphical view).

(c) Insert the complete set of data that you have come up with and show the insert statements used.

```
drop table Goodwill_Branch;
create table Goodwill_Branch
(Branchid varchar(20) primary key,
BranchLocation varchar(30) not null,
City varchar(25) not null,
Zipcode varchar(10),
state varchar(20) not null);
```

```
drop table Goodwill_Employee;
create table Goodwill_Employee
(
Employeeid varchar(20) primary key,
Firstname varchar(25) NOT null,
LastName varchar(25) not null,
Position varchar(25),
BranchId varchar(20) references Goodwill_branch(BranchId),
HireDate date not null,
Phone varchar(10));
```

```
drop table Goodwill_Donor;
Create table Goodwill_Donor
(DonorID varchar(20) primary key,
DonorName varchar(30),
ItemReceived varchar(100),
DateReceived Date);
```

Dependencies X Dependents X Processes X Goodwill/postgreu... X postgres/postgreu... X Goodwill/postgreuser

Goodwill/postgreuser@goodwillstore.postgres.database.az...

No limit

Query Query History

```
1 create table Goodwill_Branch
2 (Branchid int primary key,
3  BranchLocation varchar(30),
4  City varchar(25),
5  Zipcode varchar(10),
6  state varchar(20));
```

Data Output Messages Notifications

CREATE TABLE

Query returned successfully in 248 msec.

Now I'm following this option in entire table creation and insertion so I'll just attach a code.

```
--drop table Goodwill_Branch;

create table Goodwill_Branch
(BranchId varchar(20) primary key,
BranchLocation varchar(30) not null,
City varchar(25) not null,
Zipcode varchar(10),
state varchar(20) not null);
```

```
insert into Goodwill_Branch(BranchId, BranchLocation, City, Zipcode, state)
values('GW1','5526 Renner Rd','Columbus','43226','OH');

insert into Goodwill_Branch(BranchId, BranchLocation, City, Zipcode, state)
values('GW2','2550 North High St','Columbus','43202','OH');

insert into Goodwill_Branch(BranchId, BranchLocation, City, Zipcode, state)
values('GW3','608 Main St','Hilliard','43026','OH');
```

```
--drop table Goodwill_Employee;

create table Goodwill_Employee
(
Employeeid varchar(20) primary key,
Firstname varchar(25) NOT null,
LastName varchar(25) not null,
Position varchar(25),
BranchId varchar(20) references Goodwill_branch(BranchId),
HireDate date not null,
Phone varchar(10));
```

```
insert into Goodwill_Employee(Employeeid,FirstName,LastName,Position, BranchID,  
HireDate,Phone)
```

```
values ('Emp1','John','Johnson','Manager','GW2','03-02-2021','7324290374');
```

```
insert into Goodwill_Employee(Employeeid, FirstName, LastName, Position, BranchId,  
HireDate, Phone)
```

```
values ('Emp2','Joshua','Soboti','Cashier','GW1','04-02-2024','6142343454');
```

```
insert into Goodwill_Employee(Employeeid, FirstName, LastName, Position, BranchId,  
HireDate, Phone)
```

```
values('Emp3','Ashlee','Nimmo','Cashier','GW3','05-06-2025','7322348992');
```

```
--drop table Goodwill_Donor;
```

```
Create table Goodwill_Donor
```

```
(DonorID varchar(20) primary key,
```

```
DonorName varchar(30),
```

```
ItemReceived varchar(100),
```

```
DateReceived Date);
```

```
insert into Goodwill_Donor(DonorId, DonorName, ItemReceived, DateReceived)
```

```
values('Donor01','John','Clothes','03-02-2023');
```

```
insert into Goodwill_Donor(DonorID, DonorName, ItemReceived, DateReceived)
```

```
values('Donor02','Kelly Mcclure','Furniture','04-05-2021');
```

```
insert into Goodwill_Donor(DonorId, DonorName, ItemReceived, DateReceived)
```

```
values('Donor03','Jessica Mendez','miscellaneous','05-03-2020');
```

```
insert into Goodwill_Donor(DonorId, DonorName, ItemReceived, DateReceived)
```

```
values('Donor4','Amber Hammer','clothes','06-02-2024');
```

```
--drop table Goodwill_Warehouse;

create table Goodwill_Warehouse
(
Datawarehouseid varchar(20) primary key,
Location varchar(20) not null,
Capacity int,
ProcessingStaffcount int);
```

```
insert into Goodwill_Warehouse(Datawarehouseid,location,capacity,processingstaffcount)
values('DW01','Columbus',1000,40);

insert into Goodwill_Warehouse(Datawarehouseid,location,capacity,processingstaffcount)
values('DW02','Michigan',1000,50);
```

```
--drop table Goodwill_Donation;

create table Goodwill_Donation
(
DonationID varchar(20) primary key,
ItemReceived varchar(50) not null,
DateReceived date not null,
Quantity int not null,
donationcondition varchar(30) CHECK (donationcondition IN
('Good','Fair','Poor','Excellent')),
branchid varchar(20) references Goodwill_Branch(branchId)
);
```

```
insert into Goodwill_Donation(DonationId, ItemReceived, DateReceived, Quantity,  
Donationcondition,branchid)
```

```
values('Donation01','Clothes','03-04-2023',10,'Good','GW1');
```

```
insert into Goodwill_Donation(DonationId, ItemReceived, DateReceived, Quantity,  
Donationcondition, branchid)
```

```
values('Donation02','Furniture','02-03-2023',5,'Fair','GW2');
```

```
insert into Goodwill_donation(Donationid, itemreceived, datereceived, quantity,  
donationcondition, branchid)
```

```
values('Donation03','jackets','04-03-2024',3,'Excellent','GW3');
```

```
--Drop table Goodwill_Item;
```

```
create table Goodwill_Item
```

```
(ItemId varchar(20) primary key,
```

```
Category varchar(30) not null,
```

```
Subcategory varchar(20),
```

```
Price decimal(10,2),
```

```
Status varchar(20) CHECK (status IN ('In Stock','Sold','Recycled')),
```

```
DatePriced date,
```

```
BranchId varchar(20) references Goodwill_Branch(BranchID));
```

```
insert into Goodwill_Item(ItemId, Category, Subcategory, Price, Status, DatePriced,
BranchID)
```

```
values('It01','Clothing','MensApparel',10,'In Stock','01-03-2025','GW1');
```

```
insert into Goodwill_Item(ItemId, Category, Subcategory, Price, Status, DatePriced,
BranchID)
```

```
values('It02','couch','Furniture',200,'Sold','03-02-2023','GW2');
```

```
insert into Goodwill_Item(ItemId, Category, Subcategory, Price, Status, DatePriced,
BranchID)
```

```
values('It03','Bed','Furniture',300,'In Stock','02-03-2024','GW3');
```

```
--Drop table Goodwill_Customer;
```

```
create table Goodwill_Customer
```

```
(CustomerId varchar(20) primary key,
```

```
PurchaseHistory date,
```

```
ItemPurchased varchar(30) references Goodwill_Item(itemId),
```

```
TransactionDetails decimal(12,2));
```



```
insert into Goodwill_Customer(CustomerId, PurchaseHistory, ItemPurchased,  
TransactionDetails)
```

```
values('cust01','01-02-2024','It01',30);
```

```
insert into Goodwill_Customer(CustomerId, PurchaseHistory, ItemPurchased,  
TransactionDetails)
```

```
values('cust02','02-04-2025','It02',40);
```

```
insert into Goodwill_Customer(CustomerId, PurchaseHistory, ItemPurchased,  
TransactionDetails)
```

```
values('cust03','02-05-2023','It03',60);
```

```
--Drop table Goodwill_Transaction;
```

```
create table Goodwill_Transaction
```

```
(TransactionID varchar(20) primary key,
```

```
DateOfSale date not null,
```

```
Totalamount decimal(12,2) not null,
```

```
unitPRice decimal(10,2) not null,
```

```
discount decimal(10,2),
```

```
PaymentMethod varchar(20) check (PaymentMethod IN ('Visa','Master')),
```

```
Quantity int,
```

```
CustomerId varchar(20) references Goodwill_Customer(CustomerId),
```

```
BranchID varchar(20) references Goodwill_Branch(BranchID));
```

```
insert into Goodwill_Transaction(TransactionId, DateOfSale, TotalAmount, unitPrice,  
discount, PaymentMethod, Quantity, CustomerId, BranchId)
```

```
values('Tr01','04-03-2025',400.00,300.00,5,'Visa',3,'cust01','GW1');
```

```
insert into Goodwill_Transaction(TransactionId, DateofSale, TotalAmount, unitprice,  
discount, PaymentMethod, Quantity, CustomerId, BranchID)
```

```
values('Tr02','05-02-2025',500.00,350.00,10,'Master',4,'cust02','GW2');
```

```
--Drop table Goodwill_Sponsor;
```

```
create table Goodwill_Sponsor
```

```
(SponsorID varchar(20) primary key,
```

```
SponsorName varchar(30),
```

```
SponsorType varchar(25));
```

```
insert into Goodwill_Sponsor(SponsorID, SponsorName, SponsorType)
```

```
values('SP1','Amazon','Targeted sponsors');
```

```
insert into Goodwill_Sponsor(SponsorId, SponsorName, SponsorType)
```

```
values('SP2','Comcast','Enterprise solution');
```

```
--Drop table Goodwill_Program;

create table Goodwill_Program
(ProgramID varchar(20) primary key,
ProgramName varchar(20),
Description varchar(40),
Targetbeneficiaries varchar(50),
fundingsource varchar(20) references Goodwill_Sponsor(SponsorID));

insert into Goodwill_Program(ProgramId, ProgramName, Description,
TargetBeneficiaries,fundingsource)
values('PR01','Job training','','Job Seekers','SP1');

insert into Goodwill_Program(ProgramID, PProgramName, Description, TargetBeneficiaries,
fundingsource)
values('PR02','Career Services','','Job Seekers','SP2');

--drop table Goodwill_Itemsubcategory;
```

```
create table Goodwill_ItemSubcategory
(Subcategoryid varchar(20) primary key,
SubCategorytype varchar(30),
ItemDetails varchar(200));
```

```
insert into Goodwill_ItemSubcategory(subcategoryid, subcategorytype, itemdetails)
values('sub01','mens apparel','clothes-jackets-pants');
```

```
insert into Goodwill_ItemSubcategory(subcategoryid, subcategorytype, itemdetails)
values('sub02','furniture','couch-sofa-chair-reclying sofas-dining table-tv stand-drawers');
```

```
insert into Goodwill_ItemSubcategory(subcategoryid, subcategorytype, itemdetails)
values('sub03','womens apparel','clothes-blouse-jackets-pants');
```

```
--creating junction tables
```

```
--drop table Goodwill_TransactionProgram;

create table Goodwill_TransactionProgram
(
AllocationID varchar(20),
ProgramId varchar(20) references Goodwill_Program(ProgramID),
TransactionID varchar(20) references Goodwill_Transaction(TransactionID),
Allocationamount varchar(20) not null,
allocationdate varchar(20) not null);

insert into Goodwill_TransactionProgram(AllocationID, ProgramID, TransactionID,
AllocationAmount, AllocationDate)
values('A01','PR01','Tr01',450.00,'04-02-2024');

insert into Goodwill_TransactionProgram(allocationid, programid, transactionid,
allocationamount, allocationDate)
values('A02','PR02','Tr02',500.00,'05-03-2025');

--drop table Goodwill_SponsorProgram;
```

create table Goodwill_SponsorPProgram

(

 sponsorId varchar(20) references Goodwill_Sponsor(SponsorID),

 ProgramID varchar(20) references Goodwill_Program(ProgramID));

insert into Goodwill_SponsorProgram(sponsorId, ProgramID) values('SP1','PR01');

insert into Goodwill_SponsorProgram(sponsorId, ProgramId) values('SP2','PR02');

Now, I'm following select statement to fetch records from different tables.

```
select * from Goodwill_Branch;
```

Output Messages Notifications

Showing rows: 1 to 3				
branchid	branchlocation	city	zipcode	state
[PK] character varying (20)	character varying (30)	character varying (25)	character varying (10)	character varying (20)
GW1	5526 Renner Rd	Columbus	43226	OH
GW2	2550 North High St	Columbus	43202	OH
GW3	608 Main St	Hilliard	43026	OH

```
36  
37 select * from goodwill_Employee;
```

Data Output Messages Notifications

Showing rows: 1 to 3 Page No: 1 of 1							
	employeeid	firstname	lastname	position	branchid	hiredate	phone
	[PK] character varying (20)	character varying (25)	character varying (25)	character varying (25)	character varying (20)	date	character varying (10)
1	Emp1	John	Johnson	Manager	GW2	2021-03-02	7324290374
2	Emp2	Joshua	Soboti	Cashier	GW1	2024-04-02	6142343454
3	Emp3	Ashlee	Nimmo	Cashier	GW3	2025-05-06	7322348992

54

55 `select * from Goodwill_Donor;`

Data Output Messages Notifications

Showing rows: 1 to 4

	donorid [PK] character varying (20)	donorname character varying (30)	itemreceived character varying (100)	datereceived date
1	Donor01	John	Clothes	2023-03-02
2	Donor02	Kelly McClure	Furniture	2021-04-05
3	Donor03	Jessica Mendez	miscellaneous	2020-05-03
4	Donor04	Amber Hammer	clothes	2024-06-02

71

72 `select * from Goodwill_warehouse;`

73 `--drop table Goodwill_Donation;`

Data Output Messages Notifications

Showing rows: 1 to 2

	datawarehouseid [PK] character varying (20)	location character varying (20)	capacity integer	processingstaffcount integer
1	DW01	Columbus	1000	40
2	DW02	Michigan	1000	50

90

91 `select * from Goodwill_Donation;`

Data Output Messages Notifications

Showing rows: 1 to 3

	donationid [PK] character varying (20)	itemreceived character varying (50)	datereceived date	quantity integer	donationcondition character varying (30)	branchid character varying (20)
1	Donation01	Clothes	2023-03-04	10	Good	GW1
2	Donation02	Furniture	2023-02-03	5	Fair	GW2
3	Donation03	jackets	2024-04-03	3	Excellent	GW3

```
108
109 select * from Goodwill_Item;
```

Data Output Messages Notifications

Showing rows: 1 to 3 Page No: 1 of 1

	itemid [PK] character varying (20)	category character varying (30)	subcategory character varying (20)	price numeric (10,2)	status character varying (20)	date date	branchid character varying (20)
1	It02	couch	Furniture	200.00	Sold	2023-03-02	GW2
2	It03	Bed	Furniture	300.00	In Stock	2024-02-03	GW3
3	It01	Clothing	MensApparel	10.00	In Stock	2025-01-03	GW1

```
123
124 select * from Goodwill_Customer;
```

Data Output Messages Notifications

Showing rows: 1 to 3 Page No: 1 of 1

	customerid [PK] character varying (20)	purchasehistory date	item character varying (30)	transactiondetails numeric (12,2)
1	cust01	2024-01-02	It01	30.00
2	cust02	2025-02-04	It02	40.00
3	cust03	2023-02-05	It03	60.00

```
143 select * from Goodwill_Transaction;
144 --Drop table Goodwill_Sponsor;
```

Data Output Messages Notifications

Showing rows: 1 to 2 Page No: 1 of 1

	transactionid [PK] character varying (20)	date date	totalamount numeric (12,2)	unitprice numeric (10,2)	discount numeric (10,2)	paymentmethod character varying (20)	quantity integer	customerid character varying (20)	branchid character varying (20)
1	Tr01	2025-04-03	400.00	300.00	5.00	Visa	3	cust01	GW1
2	Tr02	2025-05-02	500.00	350.00	10.00	Master	4	cust02	GW2

```
155
156 select * from Goodwill_Sponsor;
157 --Drop table Goodwill_Program;
158 create table Goodwill_Program
```

Data Output Messages Notifications

	sponsorid [PK] character varying (20)	sponsorname character varying (30)	sponsortype character varying (25)
1	SP1	Amazon	Targeted sponsors
2	SP2	Comcast	Enterprise solution


```

169
170 select * from Goodwill_Program;
171 --drop table Goodwill_Itemsubcategory;

```

Data Output Messages Notifications

	programid [PK] character varying (20)	programname character varying (20)	description character varying (40)	targetbeneficiaries character varying (50)	fundingsource character varying (20)
1	PR01	Job training		Job Seekers	SP1
2	PR02	Career Services		Job Seekers	SP2

```

183
184 select * from Goodwill_ItemSubcategory;
185 --creating junction tables

```

Data Output Messages Notifications

	subcategoryid [PK] character varying (20)	subcategorytype character varying (30)	itemdetails character varying (200)
1	sub02	furniture	couch-sofa-chair-reclying sofas-dining table-tv stand-drawers
2	sub03	womens apparel	clothes-blouse-jackets-pants
3	sub01	mens apparel	clothes-jackets-pants

```

199
200 select * from Goodwill_TransactionProgram;
201 --drop table Goodwill_SponsorProgram;

```

Data Output Messages Notifications

	allocationid character varying (20)	programid character varying (20)	transactionid character varying (20)	allocationamount character varying (20)	allocationdate character varying (20)
1	A01	PR01	Tr01	450.00	04-02-2024
2	A02	PR02	Tr02	500.00	05-03-2025

209

210

```
select * from Goodwill_SponsorProgram;
```

Data Output

Messages

Notifications



	sponsorid character varying (20) 	programid character varying (20) 
1	SP1	PR01
2	SP2	PR02

Sometimes we realize after certain period that these fields are necessary and we should have it. To add a specific field, after creating a table, we can follow following steps:

```
alter table Goodwill_Itemsubcategory
```

```
add column itemid varchar(20);
```

```
alter table Goodwill_Itemsubcategory
```

```
add constraint fk_itemsid
```

```
foreign key(itemid)
```

```
references Goodwill_item(itemid)
```

```
update Goodwill_Itemsubcategory set itemid='lt01' where subcategoryid='sub01';
```

```
update Goodwill_Itemsubcategory set itemid='lt02' where subcategoryid='sub02';
```

```
update Goodwill_Itemsubcategory set itemid='lt02' where subcategoryid='sub03';
```

```
select * from Goodwill_Item;
```

7. Create a variety of SQL queries to retrieve data from one or many tables:

1. Retrieve the data from each table by using the SELECT * statement and order by PK column(s).

Show the output. Make sure you show the print screen of the complete set of rows and columns.
The rows must be ordered by the PK column(s).

```
17
18 select * from Goodwill_Branch order by BranchID;
19 --drop table Goodwill_Employee;
```

Data Output Messages Notifications

	branchid [PK] character varying (20)	branchlocation character varying (30)	city character varying (25)	zipcode character varying (10)	state character varying (20)
	GW1	5526 Renner Rd	Columbus	43226	OH
	GW2	2550 North High St	Columbus	43202	OH
	GW3	608 Main St	Hilliard	43026	OH

```
36
37 select * from goodwill_Employee order by employeeid;
```

Data Output Messages Notifications

	employeeid [PK] character varying (20)	firstname character varying (25)	lastname character varying (25)	position character varying (25)	branchid character varying (20)	hiredate date	phone character varying (10)
1	Emp1	John	Johnson	Manager	GW2	2021-03-02	7324290374
2	Emp2	Joshua	Soboti	Cashier	GW1	2024-04-02	6142343454
3	Emp3	Ashlee	Nimmo	Cashier	GW3	2025-05-06	7322348992

```
53
54
55 select * from Goodwill_Donor order by donorid;
56
```

Data Output Messages Notifications

	donorid [PK] character varying (20)	donorname character varying (30)	itemreceived character varying (100)	datereceived date
1	Donor01	John	Clothes	2023-03-02
2	Donor02	Kelly McClure	Furniture	2021-04-05
3	Donor03	Jessica Mendez	miscellaneous	2020-05-03
4	Donor4	Amber Hammer	clothes	2024-06-02

```
71
72 select * from Goodwill_warehouse order by Datawarehouseid;
73 --drop table Goodwill_Donation;
74 create table Goodwill_Donation
75 (
```

Data Output Messages Notifications

	datawarehouseid [PK] character varying (20)	location character varying (20)	capacity integer	processingstaffcount integer
	DW01	Columbus	1000	40
	DW02	Michigan	1000	50

```
90
91 select * from Goodwill_Donation order by DonationID;
```

Data Output Messages Notifications

	donationid [PK] character varying (20)	itemreceived character varying (50)	datereceived date	quantity integer	donationcondition character varying (30)	branchid character varying (20)
1	Donation01	Clothes	2023-03-04	10	Good	GW1
2	Donation02	Furniture	2023-02-03	5	Fair	GW2
3	Donation03	jackets	2024-04-03	3	Excellent	GW3

```
108
109 select * from Goodwill_Item order by ItemID;
110 --Drop table Goodwill_Customer;
```

Data Output Messages Notifications

	itemid [PK] character varying (20)	category character varying (30)	subcategory character varying (20)	price numeric (10,2)	status character varying (20)	date date	branchid character varying (20)
1	It01	Clothing	MensApparel	10.00	In Stock	2025-01-03	GW1
2	It02	couch	Furniture	200.00	Sold	2023-03-02	GW2
3	It03	Bed	Furniture	300.00	In Stock	2024-02-03	GW3

```
81
82 select * from Goodwill_Customer order by CustomerID;
83 --Drop table Goodwill_Transaction;
```

Data Output Messages Notifications

	customerid [PK] character varying (20)	purchasehistory date	itempurchased character varying (30)	transactiondetails numeric (12,2)
1	cust01	2024-01-02	It01	30.00
2	cust02	2025-02-04	It02	40.00
3	cust03	2023-02-05	It03	60.00

```
93 BranchID varchar(20) references Goodwill_Branch(BranchID));
94 select * from Goodwill_Transaction order by TransactionID;
95 --Drop table Goodwill_Sponsor;
```

Data Output Messages Notifications

Showing rows: 1 to 2 Page No: 1 of 1									
	transactionid [PK] character varying (20)	dateofsale date	totalamount numeric (12,2)	unitprice numeric (10,2)	discount numeric (10,2)	paymentmethod character varying (20)	quantity integer	customerid character varying (20)	branchid character varying (20)
1	Tr01	2025-04-03	400.00	300.00	5.00	Visa	3	cust01	GW1
2	Tr02	2025-05-02	500.00	350.00	10.00	Master	4	cust02	GW2

```
99 sponsortype varchar(25));
100
101 select * from Goodwill_Sponsor order by SponsorID;
```

Data Output Messages Notifications

	sponsorid [PK] character varying (20)	sponsorname character varying (30)	sponsortype character varying (25)
1	SP1	Amazon	Targeted sponsors
2	SP2	Comcast	Enterprise solution

```
109
110 select * from Goodwill_Program order by ProgramID;
```

Data Output Messages Notifications

	programid [PK] character varying (20)	programname character varying (20)	description character varying (40)	targetbeneficiaries character varying (50)	fundingsource character varying (20)
1	PR01	Job training		Job Seekers	SP1
2	PR02	Career Services		Job Seekers	SP2

```
118
119 select * from Goodwill_ItemSubcategory order by SubcategoryID;
```

Data Output Messages Notifications

	subcategoryid [PK] character varying (20)	subcategorytype character varying (30)	itemdetails character varying (200)
1	sub01	mens apparel	clothes-jackets-pants
2	sub02	furniture	couch-sofa-chair-reclying sofas-dining table-tv stand-drawers
3	sub03	womens apparel	clothes-blouse-jackets-pants

```
128
129 select * from Goodwill_TransactionProgram order by allocationid;
```

Data Output Messages Notifications

	allocationid character varying (20)	programid character varying (20)	transactionid character varying (20)	allocationamount character varying (20)	allocationdate character varying (20)
1	A01	PR01	Tr01	450.00	04-02-2024
2	A02	PR02	Tr02	500.00	05-03-2025

136

137

select * from Goodwill_SponsorProgram order by sponsorID;

Data Output

Messages

Notifications

≡

📄

▼

📋

▼

🗑️

🗄️

⬇️

📈

SQL

	sponsorid character varying (20) 🔒	programid character varying (20) 🔒
1	SP1	PR01
2	SP2	PR02

2. Write an SQL involving the junction table and two other related tables. You must use the INNER JOIN to connect with all three tables. The database that you created must be included in your SQL queries.

```
select s.SponsorID, s.SponsorName, p.ProgramName, p.Targetbeneficiaries,
p.fundingsource
```

```
from Goodwill_Sponsor s inner join Goodwill_SponsorProgram sp
```

```
on s.SponsorID=sp.SponsorID
```

```
inner join Goodwill_Program p
```

```
on p.ProgramID=sp.ProgramID
```

```
order by s.SponsorID
```

99

100

101

102

103

104

105

106

select s.SponsorID, s.SponsorName, p.ProgramName, p.Targetbeneficiaries, p.fundingsource

from Goodwill_Sponsor s inner join Goodwill_SponsorProgram sp

on s.SponsorID=sp.SponsorID

inner join Goodwill_Program p

on p.ProgramID=sp.ProgramID

order by s.SponsorID

Data Output

Messages

Notifications

≡

📄

▼

📋

▼

🗑️

🗄️

⬇️

📈

SQL

Showing rows: 1 to 2 Page

	sponsorid character varying (20) 🔒	sponsorname character varying (30) 🔒	programname character varying (20) 🔒	targetbeneficiaries character varying (50) 🔒	fundingsource character varying (20) 🔒
1	SP1	Amazon	Job training	Job Seekers	SP1
2	SP2	Comcast	Career Services	Job Seekers	SP2

3. Write an SQL statement by including two or more tables and using the LEFT OUTER JOIN. Show the results and sort the results by key field(s). Interpret the results compared to what an INNER JOIN does.

```
select c.purchasehistory, c.itemurchased, c.transactiondetails, t.totalamount,
t.paymentmethod,c.customerid,b.branchid from Goodwill_Customer c
left outer join Goodwill_Transaction t
on t.customerid=c.customerid
left outer join Goodwill_Branch b
on t.branchid=b.branchid
order by c.CustomerId;
```

107
108
109
110
111
112
113
114

```
select c.purchasehistory, c.itemurchased, c.transactiondetails, t.totalamount,
t.paymentmethod,c.customerid,b.branchid from Goodwill_Customer c
left outer join Goodwill_Transaction t
on t.customerid=c.customerid
left outer join Goodwill_Branch b
on t.branchid=b.branchid
order by c.CustomerId;
```

Data Output Messages Notifications

Showing rows: 1 to 3 Page No: 1 of 1

	purchasehistory date	itemurchased character varying (30)	transactiondetails numeric (12,2)	totalamount numeric (12,2)	paymentmethod character varying (20)	customerid character varying (20)	branchid character varying (20)
1	2024-01-02	It01	30.00	400.00	Visa	cust01	GW1
2	2025-02-04	It02	40.00	500.00	Master	cust02	GW2
3	2023-02-05	It03	60.00	[null]	[null]	cust03	[null]

98
99
100
101
102
103
104
105

```
select c.purchasehistory, c.itemurchased, c.transactiondetails, t.totalamount,
t.paymentmethod,c.customerid,b.branchid from Goodwill_Customer c
inner join Goodwill_Transaction t
on t.customerid=c.customerid
inner join Goodwill_Branch b
on t.branchid=b.branchid;
```

Data Output Messages Notifications

Showing rows: 1 to 2 Page No: 1 of 1

	purchasehistory date	itemurchased character varying (30)	transactiondetails numeric (12,2)	totalamount numeric (12,2)	paymentmethod character varying (20)	customerid character varying (20)	branchid character varying (20)
1	2024-01-02	It01	30.00	400.00	Visa	cust01	GW1
2	2025-02-04	It02	40.00	500.00	Master	cust02	GW2

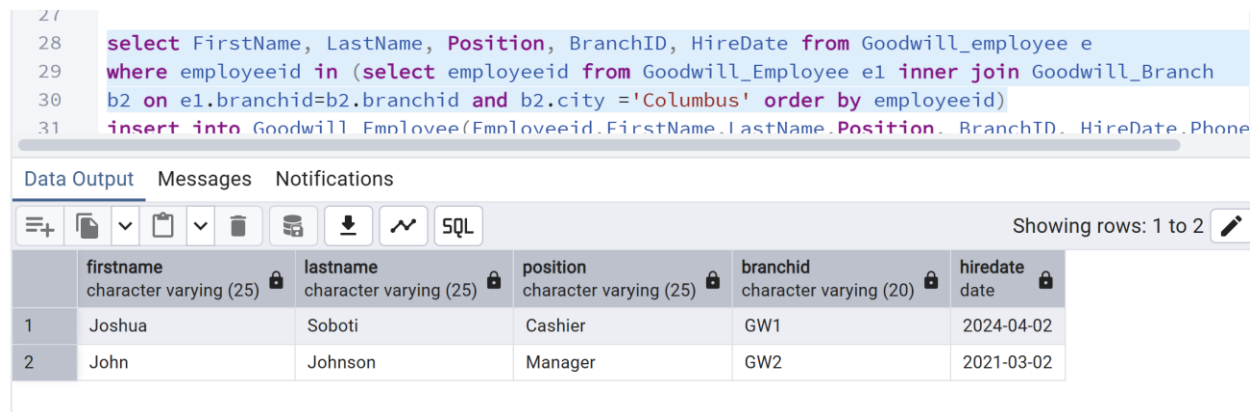
Inner join will display only those records which are common in three tables when left join will display those records which are present in the left table.

4. Write a single-row subquery. Show the results and sort the results by key field(s). Interpret the output.

```
select FirstName, LastName, Position, BranchID, HireDate from Goodwill_employee e
```

```
where employeeid in (select employeeid from Goodwill_Employee e1 inner join  
Goodwill_Branch
```

```
b2 on e1.branchid=b2.branchid and b2.city ='Columbus' order by employeeid)
```



The screenshot shows a SQL IDE interface. The top pane displays a SQL query with line numbers 27 to 31. The query selects employee details from the Goodwill_employee table, filtering for employees in the Columbus branch. The bottom pane shows the 'Data Output' tab with a table of 2 rows. The table has columns: firstname, lastname, position, branchid, and hiredate. The first row shows Joshua Soboti, Cashier, GW1, hired on 2024-04-02. The second row shows John Johnson, Manager, GW2, hired on 2021-03-02.

```
27  
28 select FirstName, LastName, Position, BranchID, HireDate from Goodwill_employee e  
29 where employeeid in (select employeeid from Goodwill_Employee e1 inner join Goodwill_Branch  
30 b2 on e1.branchid=b2.branchid and b2.city ='Columbus' order by employeeid)  
31 insert into Goodwill_Employee(Employeeid,FirstName,LastName,Position, BranchID, HireDate, Phone
```

	firstname character varying (25)	lastname character varying (25)	position character varying (25)	branchid character varying (20)	hiredate date
1	Joshua	Soboti	Cashier	GW1	2024-04-02
2	John	Johnson	Manager	GW2	2021-03-02

5. Write a multiple-row subquery. Show the results and sort the results by key field(s). Interpret the output.

```
select customerid, purchasehistory, itempurchased
```

```
from Goodwill_Customer c where c.itempurchased in
```

```
(select itemid from Goodwill_item i where status='In Stock'
```

```
and itemid in (select itemid from Goodwill_itemsubcategory its where
```

```
subcategorytype in ('mens apparel','womens apparel') ))
```

```

84
85 select customerid, purchasehistory, itempurchased
86 from Goodwill_Customer c where c.itempurchased in
87 (select itemid from Goodwill_item i where status='In Stock'
88 and itemid in (select itemid from Goodwill_itemsubcategory its where
89 subcategorytype in ('mens apparel','womens apparel') ))

```

Data Output Messages Notifications

Showing

	customerid [PK] character varying (20)	purchasehistory date	itempurchased character varying (30)
1	cust01	2024-01-02	It01
2	cust03	2023-02-05	It03

6. Write an SQL to aggregate the results by using multiple columns in the SELECT clause. Interpret the output.

```

select i.itemid, category, subcategory, price, status from Goodwill_Item i
inner join Goodwill_customer c
on c.itempurchased=i.itemid
order by i.itemid

```

```

178
179 select i.itemid, category, subcategory, price, status from Goodwill_Item i
180 inner join Goodwill_customer c
181 on c.itempurchased=i.itemid

```

Data Output Messages Notifications

Showing rows: 1 to 3

	itemid [PK] character varying (20)	category character varying (30)	subcategory character varying (20)	price numeric (10,2)	status character varying (20)
1	It01	Clothing	MensApparel	10.00	In Stock
2	It02	couch	Furniture	200.00	Sold
3	It03	Bed	Furniture	300.00	In Stock

7. Write a subquery using the NOT IN operator. Show the results and sort the results by key field(s). Interpret the output.

```

select FirstName, LastName, Position, BranchID, HireDate from Goodwill_employee e
where employeeid not in (select employeeid from Goodwill_Employee e1
inner join Goodwill_Branch
b2 on e1.branchid=b2.branchid and b2.city ='Columbus' order by employeeid)

```

167

168

169

170

171

```

select FirstName, LastName, Position, BranchID, HireDate from Goodwill_employee e
where employeeid not in (select employeeid from Goodwill_Employee e1
inner join Goodwill_Branch
b2 on e1.branchid=b2.branchid and b2.city ='Columbus' order by employeeid)

```

Data Output

Messages

Notifications

≡+

📄

▼

📋

▼

🗑️

🗄️

⬇️

📈

SQL

Showing rows: 1 to 1 

	firstname character varying (25) 	lastname character varying (25) 	position character varying (25) 	branchid character varying (20) 	hiredate date 
1	Ashlee	Nimmo	Cashier	GW3	2025-05-06

8. Write a query using a CASE statement. Show the results and sort the results by key field(s). Interpret the output.

```
select itemid, category, subcategory,
```

```
CASE
```

```
when status ='Sold' then 'out of stock'
```

```
when status='In Stock' then 'In Stock'
```

```
when status='Recycled' then 'Recycled'
```

```
else 'Empty'
```

```
end as Item_Status
```

```
from Goodwill_item;
```

```

177 select itemid, category, subcategory,
178 CASE
179     when status = 'Sold' then 'out of stock'
180     when status = 'In Stock' then 'In Stock'
181     when status = 'Recycled' then 'Recycled'
182     else 'Empty'
183 end as Item_Status
184 from Goodwill_item;
185
186 select i.itemid, category, subcategory, price, status from Goodwill_Item i

```

Data Output Messages Notifications

Showing rows: 1 to 3

	itemid [PK] character varying (20)	category character varying (30)	subcategory character varying (20)	item_status text
1	It02	couch	Furniture	out of stock
2	It03	Bed	Furniture	In Stock
3	It01	Clothing	MensApparel	In Stock

9. Write a query using the NOT EXISTS operator. Show the results and sort the results by key field(s). Interpret the output.

```

select FirstName, LastName, Position, BranchID, HireDate from Goodwill_employee e
where NOT EXISTS (select 1 from Goodwill_Branch
b2 where b2.city = 'Hilliard' and b2.branchid=e.branchid order by e.employeeid);

```

```

167
168 select FirstName, LastName, Position, BranchID, HireDate from Goodwill_employee e
169 where NOT EXISTS (select 1 from Goodwill_Branch
170 b2 where b2.city = 'Hilliard' and b2.branchid=e.branchid order by e.employeeid);
171

```

Data Output Messages Notifications

Showing rows: 1 to 2

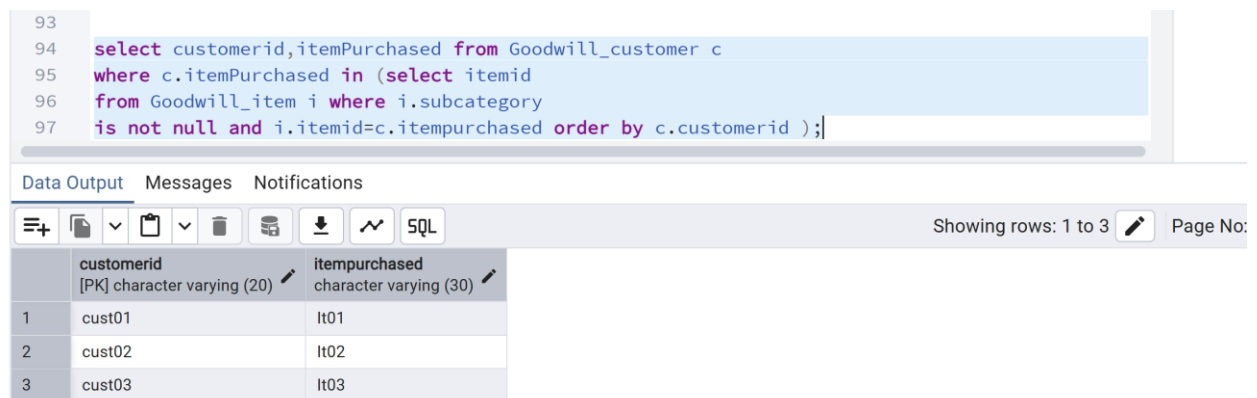
	firstname character varying (25)	lastname character varying (25)	position character varying (25)	branchid character varying (20)	hiredate date
1	John	Johnson	Manager	GW2	2021-03-02
2	Joshua	Soboti	Cashier	GW1	2024-04-02

10. Write a subquery using the NOT NULL operator in the inner query. Show the results and sort the results by key field(s). Interpret the output.

To see the proper output, I've added a null row here.

```
insert into Goodwill_Item(ItemId, Category, Subcategory, Price, Status, DatePriced,
BranchID)
values('It04','Clothing',' ',100,'In Stock','01-08-2024','GW2');
```

```
select customerid,itemPurchased from Goodwill_customer c
where c.itemPurchased in (select itemid
from Goodwill_item i where i.subcategory
is not null and i.itemid=c.itempurchased order by c.customerid );
```



The screenshot shows a database query editor with a SQL query and its results. The query is as follows:

```
select customerid,itemPurchased from Goodwill_customer c
where c.itemPurchased in (select itemid
from Goodwill_item i where i.subcategory
is not null and i.itemid=c.itempurchased order by c.customerid );
```

Below the query, there is a table with the following data:

	customerid [PK] character varying (20)	itempurchased character varying (30)
1	cust01	It01
2	cust02	It02
3	cust03	It03

SUBMISSION REQUIREMENT:

Submit the final report with details of each phase of the project, starting from the problem statement. That means you must submit all the above 7 steps (including the requirements of Update 1, Update 2, and steps 5, 6, and 7). Provide a summary (two paragraphs) of your work at the very end of the report.

Upload your submission to the Week 7 Final Project Report Dropbox in D2L.

Conclusion:

By this approach, Goodwill store can store data in an organized manner. They're able to see revenue and inventory records. They can segregate data by state, city and zip code as well. They can also identify what items need to be polished. By this way, they can keep track of their inventory, employees, sponsors, programs, donors, donations and transactions.

This database will be accessible to each Goodwill store. It's kind of enterprise resource planning system where one system will be able to organize stores all over USA.

Future Work:

I'm planning to implement change audit log to see if any record has been either removed or modified. Also, I'm planning to implement Azure data factory pipelines to make automation insertions. Along with that, to see the inventory sale, I'll generate a few PowerBI graphs.

This was all about databases and data analysis. Going forward, we'll think about frontend side as well. The frontend is also imperative to access the system. I'll develop the frontend using either PHP or .NET application.

References:

- (1) Relational Model in DBMS - GeeksforGeeks
- (2) DBMS Normalization: 1NF, 2NF, 3NF Database Example
- (3) Hierarchical Model in DBMS - GeeksforGeeks
- (4) Hierarchical Database (Model, Advantages, Disadvantages)
- (5) Network Model in DBMS - GeeksforGeeks